



Name: Rahin Ibne Harun

Id: 23-54177-3

Sec: U

Course: Compiler Design

Assignment Lab Task-4

Task:

You are given an input file named **input.txt**. Each line contains **one statement**.

Your program must do the following:

Step A, Read and classify every line

For each line, classify it as one of these:

1. Variable Declaration
2. Function Declaration
3. Invalid

Write the classification for each line into **report.txt** in this format:

Line <n>: <classification>

Step B, Write only valid lines into a new file

Create **valid_code.txt** that contains only the lines that are classified as Variable Declaration or Function Declaration, in the same order.

Step C, Add a summary at the end of report.txt

At the end of report.txt, append:

- Total lines read
- Number of variable declarations
- Number of function declarations
- Number of invalid lines

Spaces may appear anywhere, but you can ignore spaces while checking.

1) Variable Declaration (only these forms)

Valid if it matches one of these patterns:

- int id;
- float id;
- double id;
- char id;

Where id must follow:

- first character is a letter or underscore
- remaining characters can be letters, digits, underscore

Examples:

- Valid: int x;, float _avg;, double a1;
- Invalid: int 2x;, int x y;, int x

2) Function Declaration (prototype, no parameters to keep it easy)

Valid if it matches:

- int fname();
- float fname();
- double fname();
- char fname();
- void fname();

Where fname follows the same identifier rule.

Examples:

- Valid: int sum();, void show();

- Invalid: int sum(int a);, int 3f();, int sum()

Sample input.txt

```
int x;  
float avg;  
int sum();  
void show();  
int 2x;  
int f(int a);
```

Expected report.txt

```
Line 1: Variable Declaration  
Line 2: Variable Declaration  
Line 3: Function Declaration  
Line 4: Function Declaration  
Line 5: Invalid  
Line 6: Invalid  
Summary:  
Total lines = 6  
Variable Declarations = 2  
Function Declarations = 2  
Invalid = 2
```

Expected valid_code.txt

```
int x;  
float avg;  
int sum();  
void show();
```

Answer:

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
#include <cctype>
```

```
using namespace std;
```

```
bool isIdentifier(string s)
```

```
{
```

```
    if (s.empty()) return false;
```

```

    if (!(isalpha(s[0]) || s[0] == '_')) return false;

    for (int i = 1; i < s.length(); i++)
    {
        if (!(isalnum(s[i]) || s[i] == '_')) return false;
    }

    return true;
}

bool isVariable(string line)
{
    string type, id;

    int pos = line.find(' ');

    if (pos == -1) return false;

    type = line.substr(0, pos);
    id = line.substr(pos + 1);

    if (id.back() != ';') return false;

    id.pop_back();

    if (!(type == "int" || type == "float" || type == "double" || type == "char"))
        return false;

    return isIdentifier(id);
}

bool isFunction(string line)
{
    string type;

    int pos = line.find(' ');

    if (pos == -1) return false;

    type = line.substr(0, pos);

```

```

    string rest = line.substr(pos + 1);

    if (!(type == "int" || type == "float" || type == "double" || type == "char" || type == "void"))
        return false;

    int p1 = rest.find('(');
    int p2 = rest.find(')');

    if (p1 == -1 || p2 == -1) return false;

    string fname = rest.substr(0, p1);

    if (rest.substr(p1) != "();") return false;

    return isIdentifier(fname);
}

int main()
{
    ifstream input("input.txt");
    ofstream report("report.txt");
    ofstream valid("valid_code.txt");

    if (!input.is_open())
    {
        cout << "input.txt open korte problem!" << endl;

        return 1;
    }

    string line;

    int lineNo = 0;

    int varCount = 0, funCount = 0, invalid = 0;

```

```

while (getline(input, line))
{
    lineNo++;
    if (isVariable(line))
    {
        report << "Line " << lineNo << ": Variable Declaration\n";
        valid << line << endl;
        varCount++;
    }
    else if (isFunction(line))
    {
        report << "Line " << lineNo << ": Function Declaration\n";
        valid << line << endl;
        funCount++;
    }
    else
    {
        report << "Line " << lineNo << ": Invalid\n";
        invalid++;
    }
}

report << "\nSummary:\n";
report << "Total lines = " << lineNo << endl;
report << "Variable Declarations = " << varCount << endl;
report << "Function Declarations = " << funCount << endl;
report << "Invalid = " << invalid << endl;

input.close();
report.close();

```

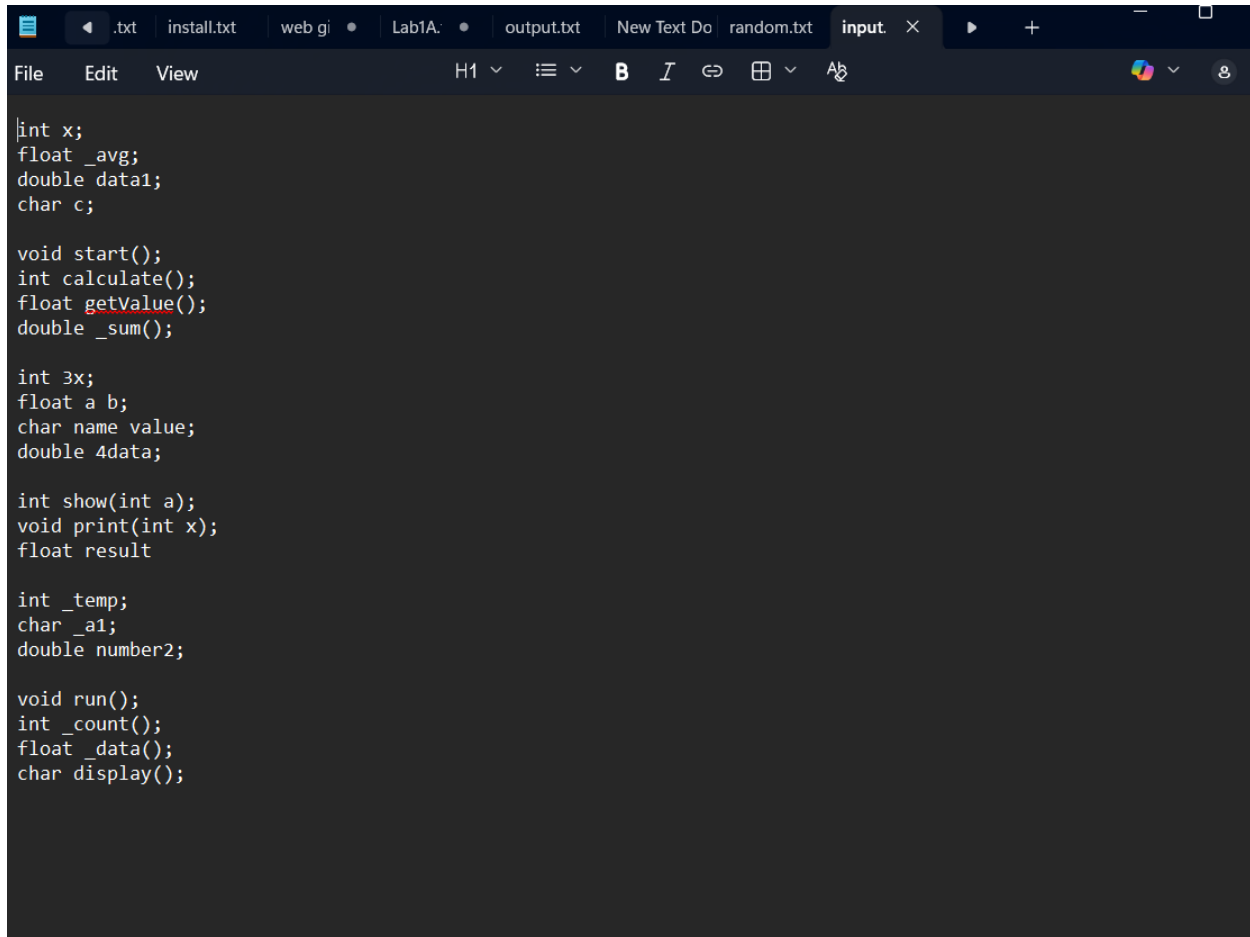
```
valid.close();

cout << "report.txt and valid_code.txt successfully created!" << endl;

return 0;

}
```

Input file ss:



The screenshot shows a code editor with a dark theme. The top bar displays several open files: .txt, install.txt, web gi, Lab1A, output.txt, New Text Do, random.txt, and input. The editor content is C++ code with the following structure:

```
int x;
float _avg;
double data1;
char c;

void start();
int calculate();
float getValue();
double _sum();

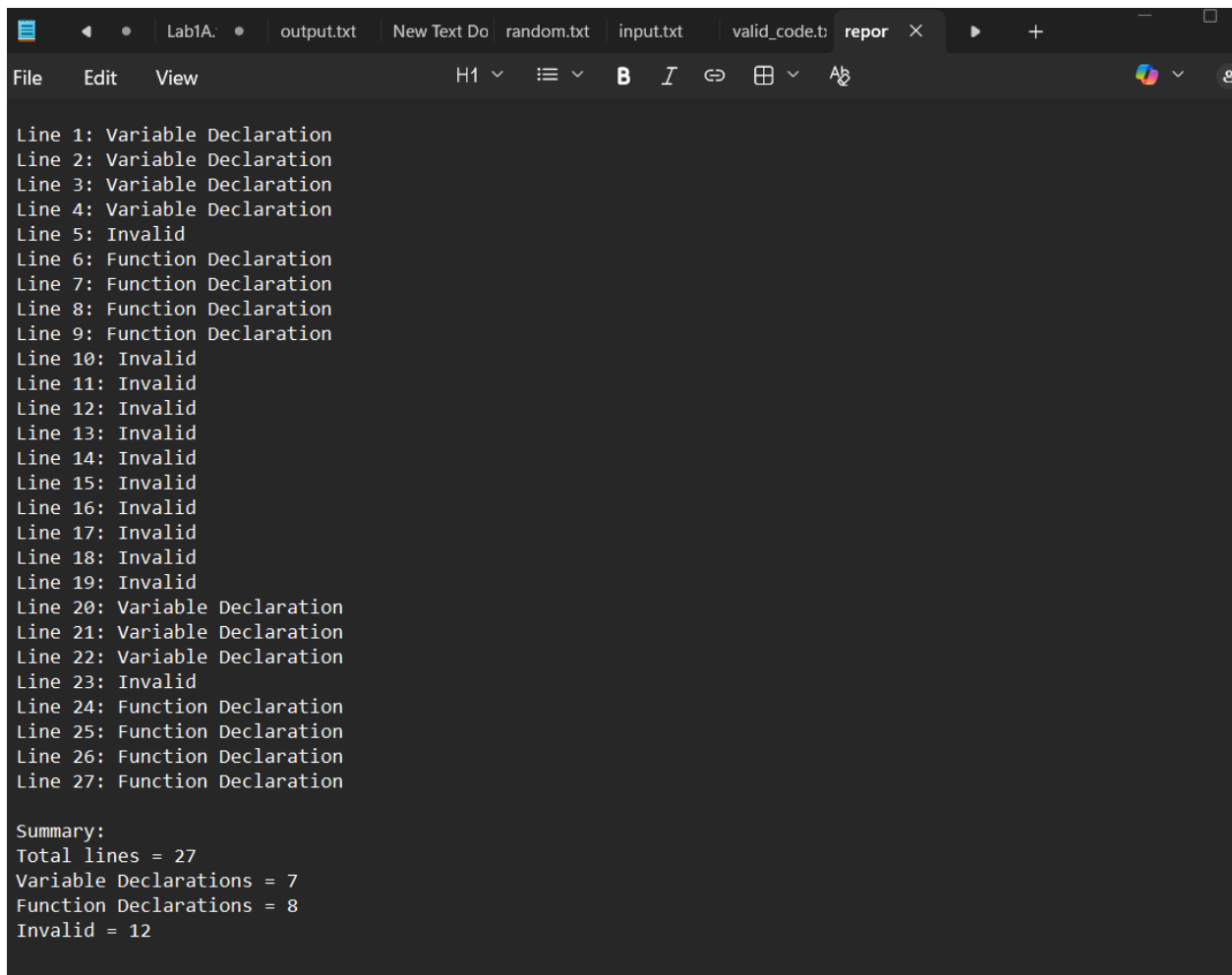
int 3x;
float a b;
char name value;
double 4data;

int show(int a);
void print(int x);
float result

int _temp;
char _a1;
double number2;

void run();
int _count();
float _data();
char display();
```

Report text file:



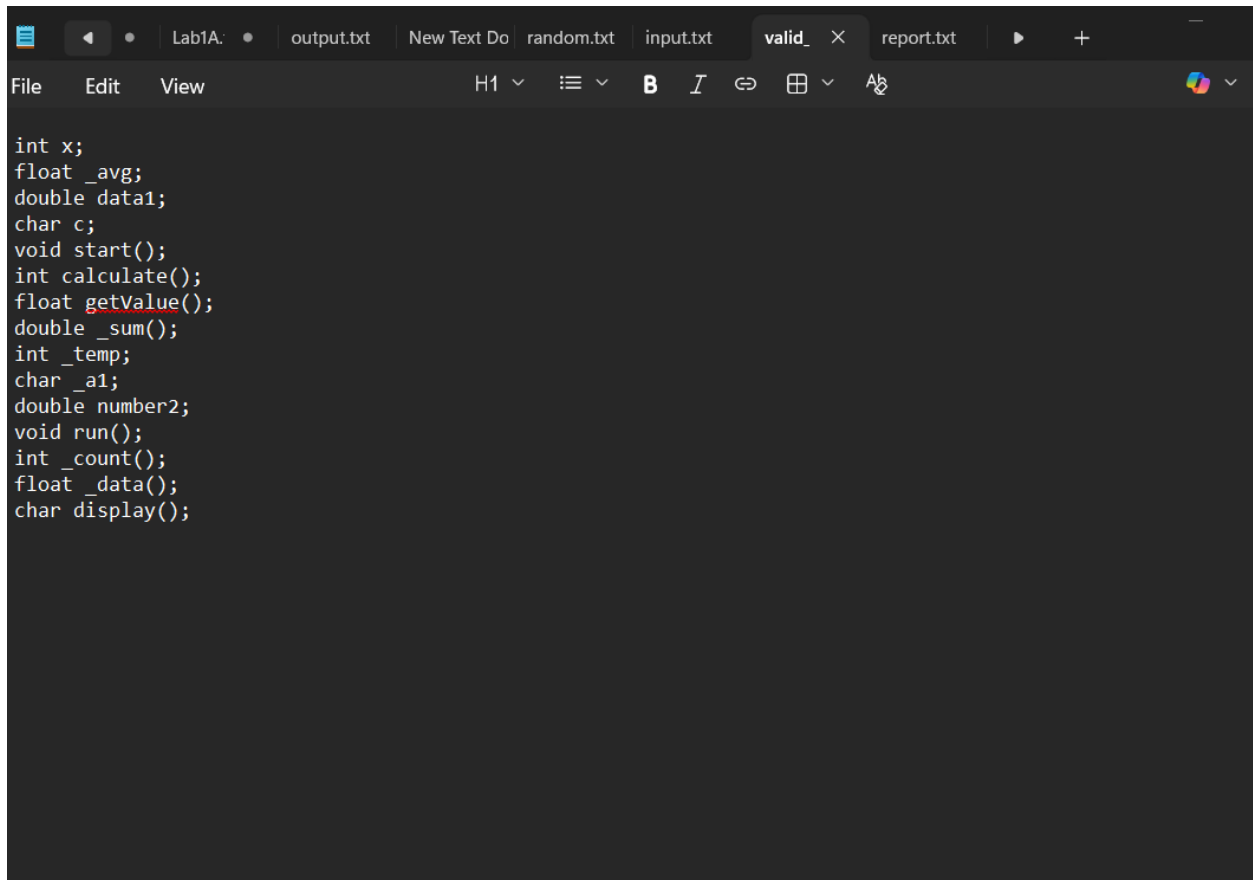
The screenshot shows a text editor window with a dark theme. The title bar at the top displays several open files: 'Lab1A:', 'output.txt', 'New Text Do', 'random.txt', 'input.txt', 'valid_code.t', and 'report'. The 'report' file is the active document. The menu bar includes 'File', 'Edit', and 'View'. The toolbar shows various editing tools like bold, italic, and undo. The main text area contains a list of 27 lines, each labeled with its line number and a classification: 'Variable Declaration', 'Function Declaration', or 'Invalid'. The lines are as follows:

- Line 1: Variable Declaration
- Line 2: Variable Declaration
- Line 3: Variable Declaration
- Line 4: Variable Declaration
- Line 5: Invalid
- Line 6: Function Declaration
- Line 7: Function Declaration
- Line 8: Function Declaration
- Line 9: Function Declaration
- Line 10: Invalid
- Line 11: Invalid
- Line 12: Invalid
- Line 13: Invalid
- Line 14: Invalid
- Line 15: Invalid
- Line 16: Invalid
- Line 17: Invalid
- Line 18: Invalid
- Line 19: Invalid
- Line 20: Variable Declaration
- Line 21: Variable Declaration
- Line 22: Variable Declaration
- Line 23: Invalid
- Line 24: Function Declaration
- Line 25: Function Declaration
- Line 26: Function Declaration
- Line 27: Function Declaration

Below the list of lines, there is a 'Summary:' section with the following statistics:

- Total lines = 27
- Variable Declarations = 7
- Function Declarations = 8
- Invalid = 12

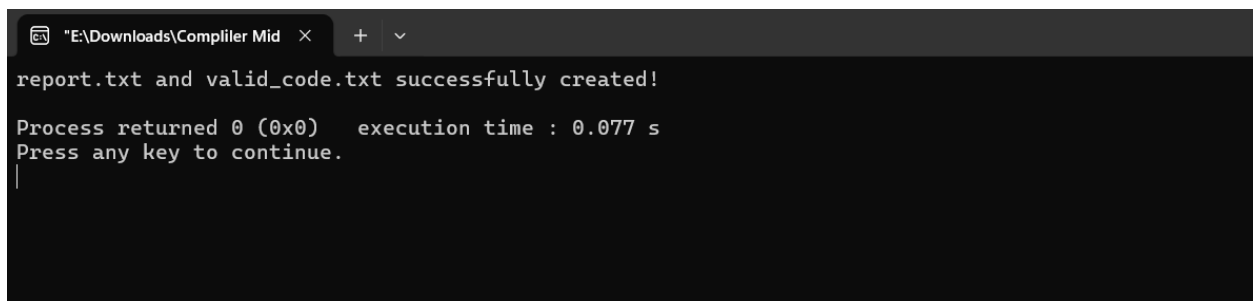
Valid_code text:



A screenshot of a code editor window with a dark theme. The title bar shows several tabs: 'Lab1A:', 'output.txt', 'New Text Do', 'random.txt', 'input.txt', 'valid_ X', 'report.txt', and a play button icon. The menu bar includes 'File', 'Edit', and 'View'. The toolbar shows 'H1', a list icon, 'B' (bold), 'I' (italic), a link icon, a grid icon, and 'Ab'. The code is written in C and includes variable declarations, function prototypes, and function definitions. The function 'getValue()' is highlighted with a red squiggly underline.

```
int x;
float _avg;
double data1;
char c;
void start();
int calculate();
float getValue();
double _sum();
int _temp;
char _a1;
double number2;
void run();
int _count();
float _data();
char display();
```

Cmd output:



A screenshot of a Windows command prompt window. The title bar shows the path 'E:\Downloads\Compiler Mid' and a plus icon. The output text indicates that 'report.txt' and 'valid_code.txt' were successfully created, the process returned 0 (0x0) with an execution time of 0.077 s, and it prompts the user to press any key to continue. A cursor is visible on the line following the prompt.

```
"E:\Downloads\Compiler Mid"
report.txt and valid_code.txt successfully created!

Process returned 0 (0x0)   execution time : 0.077 s
Press any key to continue.
|
```

//////////////////// End////////////////////